



Introduction to Data Science

Assignment 3

Instructors: **Dr. Razavi,**
Dr. Yaghoobzadeh

TAs: **Faezeh Mozaffari, Golboo Rashidi,**
Amir Naddaf

Deadline: Khordad 10th, 11:59PM



Introduction

The goal of this assignment is to work on three practical data science problems: fraud detection in transactions, predicting the number of article shares, and ranking candidates for job positions. The main focus is to follow a complete machine learning workflow, including data analysis, preprocessing, feature engineering, model selection, evaluation, and final submission generation.

Only traditional machine learning methods are allowed in this assignment. Deep learning models, pre-trained models, and AutoML tools are not permitted. Students should also carefully avoid data leakage, use suitable evaluation metrics, and make sure their results are reproducible.

Assignment Details and Instructions

Each of the three tasks in this assignment has a corresponding Quera competition. You can participate by clicking on the Quera competition link provided at the end of each task description.

What You Must Do:

- Click the competition link at the bottom of each task to join the Quera competition.
- Once your model is ready, generate predictions on the test set and save them in a CSV file. The expected submission format is described on the competition page (a CSV file with columns like id and label).
- Upload the prediction file to the Quera competition page for each task.
- The test labels are hidden — this is to prevent you from tuning your model on the test set. Avoid overfitting to the public leaderboard.
- Do NOT use deep learning models or architectures.
- Do NOT use or fine-tune pre-trained models.
- You are allowed to use traditional machine learning libraries, including but not limited to: scikit-learn, CatBoost, XGBoost, and LightGBM.
- The most critical factor in improving your model's accuracy is good data preprocessing and feature engineering. We strongly recommend investing time in understanding and transforming your data
- The use of deep neural networks or automated machine learning (AutoML) tools is not permitted.

Task 1 : Classification

Objective:

This project challenges you to develop a robust machine learning classification model to predict whether a retail transaction is fraudulent. Using a dataset containing transaction details, customer activity patterns, payment information, device and location data, merchant categories, and fraud-related behavioral indicators, the primary goal is to accurately classify the target label `fraud_flag`, where 1 signifies “Fraudulent” and 0 signifies “Legitimate”.

Process Overview:

Success in this project depends on a structured machine learning workflow that includes data exploration, preprocessing, feature engineering, model development, and evaluation.

1. **Data Exploration & Understanding:** Begin by thoroughly analyzing the retail transaction dataset. Investigate the relationships between features and the target variable `fraud_flag`. The goal is to understand which customer behaviors, transaction patterns, and contextual signals are most associated with fraudulent activity.
2. **Preprocessing & Feature Engineering:** Effective data preparation is crucial for building a high-performing model. Apply appropriate techniques such as handling missing values, encoding categorical features, scaling numerical data, managing date information, and potentially engineering new features to capture predictive signals better.
3. **Data Leakage:** One of the most important points in this task is avoiding Data Leakage. Some features may introduce future information into the model if they are not calculated correctly and strictly based only on information available before the transaction time. If leakage is used and the model achieves an unrealistically high score, the result will not be accepted and that part will receive a score of zero. Therefore, participants must be very careful during preprocessing, feature engineering, and validation to ensure that no future information or validation/test information is used during training.
4. **Model Development & Selection:** Experiment with various binary classification algorithms suitable for this task. Compare their performance using appropriate validation strategies and select the model that yields the highest **F1 Score** on your validation set.
5. **Class Imbalance cautions:** In fraud detection problems, the class distribution is usually imbalanced; in other words, fraudulent transactions are often much less frequent than legitimate transactions. Therefore, participants should pay close attention to this issue during data analysis and model evaluation. In such cases, relying only on Accuracy can be misleading, because a model may achieve a high

Accuracy by mostly predicting the majority class while still performing poorly in detecting fraudulent transactions. Therefore, the impact of class imbalance on model performance should be carefully considered during model development.

6. **Classification threshold:** Many classification models produce probability scores. For example, a model may predict that a transaction has a fraud probability of 0.73. To convert this probability into a final class label of 0 or 1, a threshold is applied, and the default threshold is usually 0.5. However, in fraud detection, a threshold of 0.5 may not be optimal. Participants can test different threshold values on the validation set and choose the one that improves model performance.
7. **Comprehensive Evaluation:** While the F1 Score is the primary metric for final scoring, The recommended classification metrics to report are: Accuracy, Precision, Recall or Sensitivity, F1 Score, Confusion Matrix, ROC-AUC, Specificity, Matthews Correlation Coefficient, Cohen's Kappa if the model produces probability outputs.

Prediction & Submission:

Your final, trained model will be used to predict the fraud_flag for a separate test dataset where the true labels are withheld.

- **Output:** Submit your predictions as a **CSV file**.
- **Format:** The CSV must contain exactly two columns:
 - id: The unique identifier for each test instance.
 - label: Your model's predicted value (0 or 1).

To participate in the competition related to this task Click on the competition link: [Quera Competition](#).

Good luck with the competition!

Task 2 : Regression

For this assignment task, you'll work with a dataset containing features of online news articles published on a content-sharing platform. Your goal is to develop a regression model that predicts the number of times an article is shared on social media.

The dataset provides a variety of attributes describing each article's content, headline, keywords, publication details, sentiment, and topics. While this is not a time series problem, you may use publication-related features (such as the day of the week or whether it is a weekend) to help explain patterns in sharing behavior.

Process Overview:

1. **Data Exploration & Understanding:** Exploratory Data Analysis (EDA) is a critical first step. You are expected to thoroughly explore the data, create visualisations, and identify relationships between features and the target. In the in-person session, you must justify your preprocessing, feature engineering, and model choices based on your EDA findings. EDA and visualisations are part of the overall evaluation.
2. **Preprocessing & Feature Engineering:** Apply appropriate preprocessing techniques, including encoding categorical variables and handling any data quality issues you identify. Since the number of available features is relatively large, careful feature selection and thoughtful feature engineering are both essential for building a well-performing model. Statistical methods can help you identify the most predictive original features, while constructing meaningful new features from the existing ones can further enhance the model's overall accuracy and robustness. Besides correlation analysis, it is highly recommended to use p-values for feature selection. This statistical test helps determine which features are significantly related to the target variable. By doing so, you can filter out irrelevant or weakly correlated features that may negatively affect model performance.
3. **Data Leakage:** One of the most important points in this task is avoiding Data Leakage. Some features may introduce future information into the model if they are not calculated correctly and strictly based only on information available before the transaction time. If leakage is used and the model achieves an unrealistically high score, the result will not be accepted and that part will receive a score of zero. Therefore, participants must be very careful during preprocessing, feature engineering, and validation to ensure that no future information or validation/test information is used during training.
4. **Regression Methods:** Next, experiment with a variety of traditional regression algorithms and select the one that performs best on the validation set. It is highly recommended to use K-Fold cross-validation to obtain a more reliable estimate of your model's performance and to tune hyperparameters. Finally, train your model and

use it to predict the number of shares for the articles in the test dataset. Make sure to experiment with different regression algorithms (e.g., Linear Regression, Decision Trees, Random Forest, or Gradient Boosting) and fine-tune the hyperparameters to achieve the best performance on the validation data.

5. **Evaluation Metrics:** To assess how well your regression model is performing, consider using Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Root Mean Squared Logarithmic Error (RMSLE), R-Squared (R² Score), Mean Absolute Percentage Error (MAPE), Mean Absolute Error (MAE) (you will show these metrics in an in-person session). Submissions are ranked by Root Mean Squared Logarithmic Error (RMSLE).

Prediction & Submission:

- **Output:** Submit your predictions as a **CSV file**.
- **Format:** The CSV must contain exactly two columns:
 - id: The unique identifier for each test instance.
 - shares: Your model's predicted value.

To participate in the competition related to this task Click on the competition link: [Quera Competition](#).

Good luck with the competition!

Task 3 : Learning to Rank

Information-retrieval systems used by recruitment platforms, search engines, and recommendation services share a structural property that distinguishes them from standard supervised-learning tasks, their predictions are evaluated within query groups rather than over the entire test set. For a job posting with one hundred applicants, what matters is the relative ordering of those one hundred candidates, not their absolute predicted scores, and not the ordering of candidates across different job postings. This family of problems is known as Learning to Rank and constitutes the algorithmic backbone of systems such as Google Search, LinkedIn Recruiter, Indeed, and Airbnb's listing ranker. In this task, you will design and implement a complete LTR pipeline for a synthetic but realistic tech-recruitment dataset.

Dataset Overview:

The dataset comprises three files:

- jobs.csv: $\approx 5,000$ job postings with 12 attributes.
- candidates.csv: $\approx 50,000$ candidate profiles.
- applications_train.csv / applications_test.csv: The training file contains the label, the test file does not. Submissions are made against applications_test.csv.

The train/test split is temporal, based on application_date:

- Training set: Applications with application_date < 2024-07-01
- Test set: Applications with application_date $\geq$ 2024-07-01

The test set includes:

- Applications for previously-seen jobs (jobs that also appear in training)
- Applications for new jobs (jobs that appear only in test set)

This tests your model's ability to rank both familiar and unfamiliar job contexts. Build features that generalize across job types.

You are provided with two years of recruitment data, TalentMatch. For every (job, candidate) pair in the training set, you are given a graded relevance label *relevance label* $\in \{0. 1. 2. 3. 4\}$ reflecting how well the candidate fits that specific posting. Your objective is to learn a scoring function $f(\text{job}, \text{candidate}) \rightarrow \mathbb{R}$ such that, when candidates are sorted in descending order of f within each job_id, the resulting ranking approximates the hidden ground-truth ranking on a held-out test set.

Each label reflects candidate_job fit:

- Label 0: Not relevant at all (poor match, recommend rejection)
- Label 1: Slightly relevant (some mismatch, marginal fit)
- Label 2: Moderately relevant (reasonable fit, worth reviewing)
- Label 3: Highly relevant (strong match, advance to interview)
- Label 4: Extremely relevant / Best match (ideal candidate, recommend hiring)

Labels are assigned per_job, not globally. A candidate may receive label 4 for one job and label 1 for another job. This is intentional and expected.

Process Overview:

1. **Data Exploration & Understanding:** Exploratory Data Analysis (EDA) is a critical first step. You are expected to thoroughly explore the data, create visualisations, and identify relationships between features and the target. In the in-person session, you must justify your preprocessing, feature engineering, and model choices based on your EDA findings. EDA and visualisations are part of the overall evaluation.
2. **Preprocessing & Data Cleaning:** Build a fully reproducible preprocessing pipeline for the dataset. The pipeline should appropriately handle missing values, categorical variable encoding, and normalization or scaling of numeric features. If target encoding is used, it must be implemented using out-of-fold (OOF) target encoding in order to avoid target leakage. You are free to use any preprocessing strategy they consider appropriate. The data is gently messy. It contains realistic inconsistencies in string normalization, multiple currency conventions for salary fields, a small fraction of missing values, and a time-based train/test split. These properties are intentional and are part of the assignment.
3. **Feature Engineering:** Design and implement at least five interaction features between candidate information and job information. Examples may include skill overlap, experience gap, salary mismatch, education compatibility, or location compatibility. Each engineered feature should be briefly explained and justified in the final report.
4. **Modeling:** You are building a scoring function $f(\text{job}, \text{candidate}) \rightarrow \text{relevance_score}$ using standard supervised learning. The final output of any model must be a continuous relevance score that can be used to rank candidates within each job_id. If a regression model is used, its predicted value can be used directly as the relevance score. If a classification model is used, participants should convert class probabilities into a single ranking score, for example by using the expected relevance. This score is then used only for sorting candidates within the same job_id. The evaluation metric directly measures ranking quality, not prediction accuracy. Standard machine learning models are completely acceptable. Ranking-oriented models such as LGBMRanker, XGBRanker, and CatBoostRanker may also be used, but they are not mandatory. The primary objective is to build an effective ranking or scoring pipeline. Implement an appropriate cross-validation strategy for the ranking task. A recommended approach

is to use GroupKFold(groups=job_id) in order to preserve the ranking context and ensure that samples belonging to the same job_id remain within the same fold. Participants are also encouraged to carefully consider data leakage, validation consistency, and fair evaluation methodology throughout the project.

5. **Evaluation Metrics:** Evaluation is done separately for each job. Candidates must only be compared with other candidates who applied to the same job_id, not with candidates from other jobs.

For each job, sort candidates by the predicted relevance score in descending order. The candidate with the highest predicted score gets rank 1. For each job, take the top 10 ranked candidates and compute:

$$DCG@10 = \sum_{i=1}^{10} \frac{2^{\frac{rel_i-1}{2}}}{i+1}$$

Here, rel_i is the true relevance label at position i. Higher relevance labels near the top receive more reward. If a job has fewer than 10 candidates, DCG@10 and IDCG@10 should be computed using all available candidates for that job. In other words, use the top min(10, number of candidates for that job) candidates.

Sort the same candidates by their true relevance labels in descending order. This creates the ideal ranking. Then compute DCG@10 for this perfect ranking.

$$IDCG@10 = DCG@10 \text{ of the ideal ranking}$$

Normalize the score for each job:

$$NDCG@10 = \frac{DCG@10}{IDCG@10}$$

A value closer to 1 means the predicted ranking is closer to the ideal ranking. If IDCG@10 is equal to zero for a job, meaning that none of the candidates for that job has a positive relevance label, NDCG@10 for that job will be defined as 0. This avoids division by zero and ensures that all jobs can be included consistently in the final average score.

Repeat the process for all jobs and average the results:

$$Score = \text{mean } NDCG@10 \text{ across all jobs}$$

For the secondary metric, the original graded relevance labels are first converted into binary labels. Candidates with labels 3 or 4 are considered relevant, while candidates with labels 0, 1, or 2 are considered non-relevant.

For each job, look at the top 5 ranked candidates. At each relevant position i, compute:

$$Precision@i = \text{number of relevant candidates in top } i / i$$

Only positions containing relevant candidates are used.

For each job, Average Precision@5, or AP@5, is computed by averaging Precision@i values only at positions where the candidate is relevant within the top 5 results.

$$AP@5 = (1 / \min(\text{number of relevant candidates}, 5)) \times \sum Precision@i \times rel_i$$

where rel_i is 1 if the candidate at position i is relevant and 0 otherwise. If a job has no relevant candidates, AP@5 for that job is defined as 0. Finally, average AP@5 across all jobs:

$$MAP@5 = \text{mean } AP@5 \text{ across all jobs}$$

NDCG@10 is the primary scoring metric, while MAP@5 is reported as a secondary metric for analyzing top-5 ranking quality.

Prediction & Submission:

- **Output:** Submit your predictions as a **CSV file**.
- **Format:** The CSV must contain exactly two columns:
 - **application_id:** The unique identifier for each test application instance.
 - **score:** Your model's predicted relevance score for that application. This score will be used to rank candidates within each job_id. Higher scores indicate candidates predicted to be more relevant for the job. (The score does not need to be a class label and does not need to be between 0 and 1. It only needs to provide a meaningful ordering of candidates within the same job posting.)

To participate in the competition related to this task Click on the competition link: [Quera Competition](#).

Good luck with the competition!

Evaluation Guidelines:

- This assignment has a total of 100 points: 60 points are allocated to your project submission and its delivery (as reviewed by the TA team), and 40 points are based on your ranking in the contest. (you can earn up to 10 points bonus)
- The contest leaderboard ranking determines how the 40 contest points are awarded. The top group receives 10% of the bonus points, the second group 9%, the third group 8%, and so on, down to the tenth group which receives 1%. Participants ranked beyond the tenth group (i.e., 11th place and onward) will lose 2% of the bonus points for each additional rank. For example, the group in 11th place will receive 38 out of 40 points.
- You will also be required to demonstrate your code live in an in-person session. During this session, your model must produce exactly the same predictions as those submitted to Quera.
- If your model's outputs during the live review differ from your Quera submission, you will receive a penalty of -100.
- If the uploaded predictions were not directly generated by your model (e.g., if you manually adjusted the results), the penalty will also be -100.
- However, in the in-person session, you will be required to report and justify other relevant metrics for each task. So make sure your code computes and logs additional evaluation metrics.

Notes

- Upload your work as a zip file in this format on the website: DS_CA3_[Std number].zip. If the project is done in a group, include all of the group members' student numbers in the name.
- Only one member must upload the work if the project is done in a group.
- We will run your code during the project delivery, so make sure your results are reproducible.
- Only traditional machine learning models are allowed; the use of any neural network models to solve the problem is strictly prohibited.